

Système d'exploitation Linux — Jour 2

Réseau, SSH, Bash, logs, sauvegarde et troubleshooting

Bachelor 3 • Systèmes, Réseaux & Cloud

Fil rouge : aujourd'hui, srv-linux01 doit devenir exploitable — puis survivre à un P1.

13 h • théorie courte • pratique intensive • troubleshooting final

Passer du serveur au service

1h30 d'exploitation/sécurité, puis 5h de TP et incident

1 EXPOSER

Comprendre IP, routes, DNS et sockets.

2 SÉCURISER

SSH par clé et moindre privilège.

3 AUTOMATISER

Bash, exit codes, cron/timers.

4 RÉSISTER

Logs, backup, restore, incident.

Aujourd'hui, "ça marche sur ma VM" ne suffit plus : il faut prouver, automatiser et restaurer.

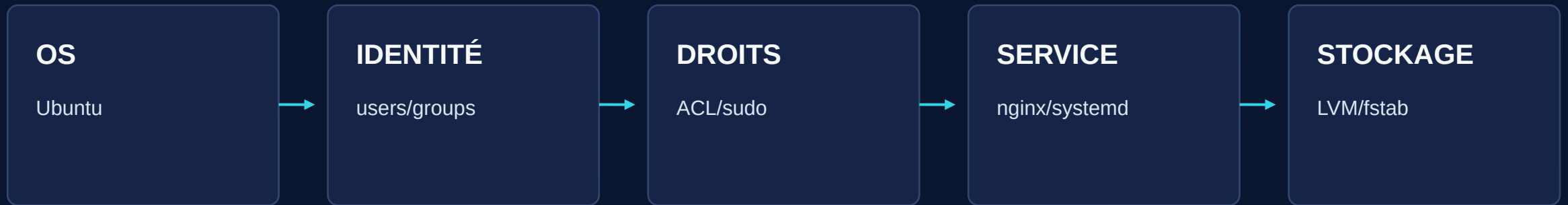
QUESTION

Quel est le point commun entre réseau, SSH, Bash, logs et backup ?

Ils servent à exploiter durablement un service, pas seulement à le démarrer.

Hier, vous avez construit les couches de base

Aujourd'hui on les exploite comme un système cohérent.



Aujourd'hui : Réseau → SSH → Healthcheck → Logs → Backup → Incident

Les pannes du Final Boss réutiliseront volontairement des couches du Jour 1.

“Nginx est démarré” n’est pas une définition de la production

Un service exploitable doit rester administrable, observable et récupérable.

ADMINISTRABLE

SSH sûr

OBSERVABLE

logs & métriques

PERSISTANT

mounts & config

AUTOMATISÉ

checks & jobs

RÉCUPÉRABLE

backup/restore

SÉCURISÉ

moindre privilège

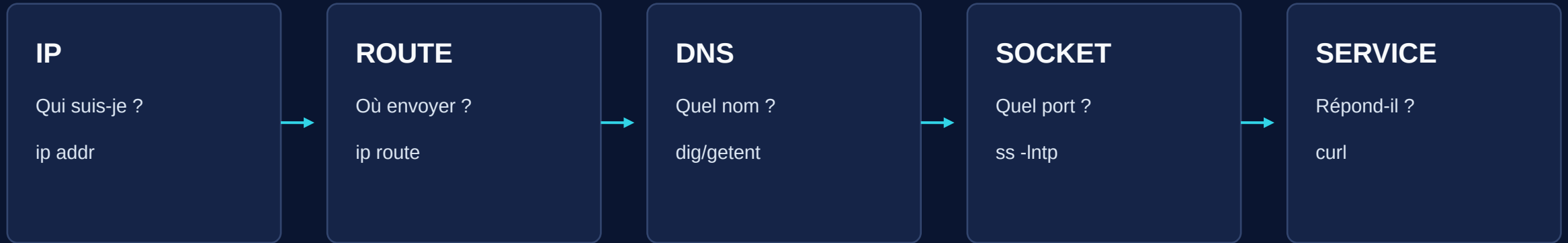
QUESTION

Un service peut-il être “UP” et pourtant inutilisable ?

Oui : port incorrect, disque plein, latence, permissions, dépendance, etc.

Diagnostiquer par couches : IP → route → DNS → socket → service

Chaque commande répond à une question précise.



Un ping réussi ne prouve ni DNS, ni TCP/80, ni HTTP.

QUESTION

ping 1.1.1.1 OK mais curl https://example.com KO : que vérifier ensuite ?

DNS, route/egress TCP, proxy, TLS... selon le message d'erreur.

Les commandes sont des sondes, pas des talismans

Associez commande ↔ question ↔ interprétation.

ip addr

Adresse / état interface

ip route

Décision de routage

resolvectl

DNS configuré

getent / dig

Résolution de nom

ss -lntp

Socket d'écoute + process

curl -I

Test applicatif HTTP

DÉMO LIVE ip addr → ip route → getent hosts → ss -lntp → curl -I http://127.0.0.1

Trois symptômes, trois raisonnements différents

Ne sautez pas directement à “le réseau est cassé”.

CAS A

ping IP OK
dig KO
curl par nom KO

→ suspecter DNS

CAS B

nginx active
ss : pas de :80

→ config / bind / autre port

CAS C

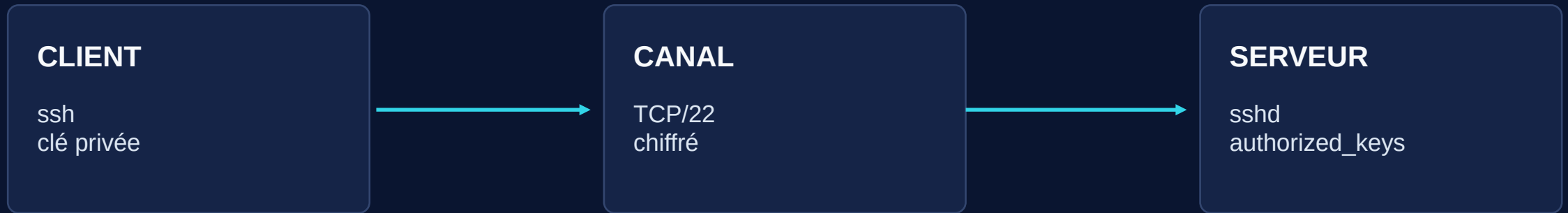
ss :80 OK
curl 127.0.0.1 = 500

→ application/config/logs

La couche réseau n’est pas “tout ce qui utilise une IP”.

SSH : canal d'administration, authentification et chiffrement

TCP/22 n'est que le début de l'histoire.



DÉMO

```
ssh user@srv-linux01
ss -lntp | grep :22
systemctl status ssh
```

QUESTION

systemctl status ssh = active. Est-ce que cela prouve que vous pourrez vous connecter ?

Non : il faut vérifier socket, réseau, auth, policy, clé, etc.

Mot de passe vs clé publique

Le serveur stocke une clé publique ; votre clé privée ne doit jamais quitter le client.

MOT DE PASSE

- Secret partagé / mémorisable
- Réutilisation fréquente
- Exposé aux tentatives de bruteforce
- Peut rester utile en bootstrap contrôlé

CLÉ SSH

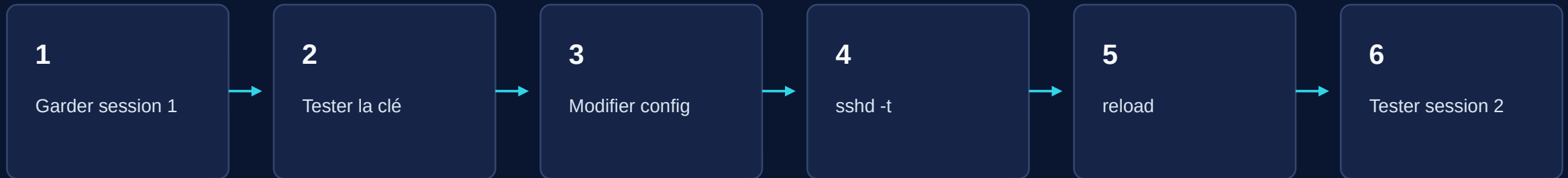
- Privée côté client
- Publique côté serveur
- Facile à automatiser / révoquer
- Meilleure base pour administration

EXEMPLE

```
ssh-keygen -t ed25519  
ssh-copy-id user@srv-linux01
```

Durcir sans se verrouiller dehors

Le changement le plus sécurisé est celui qui prévoit son propre rollback.



SAFE CHANGE

```
sudo /usr/sbin/sshd -t  
sudo systemctl reload ssh  
ssh -o PreferredAuthentications=publickey user@srv-linux01
```

PermitRootLogin no devient obligatoire. PasswordAuthentication no seulement après preuve de la clé.

Automatiser : transformer des observations en un résultat exploitable

Un script d'exploitation doit être lisible par l'humain ET par une machine.

BASH

```
#!/usr/bin/env bash
SERVICE="nginx"

if systemctl is-active --quiet "$SERVICE"; then
    echo "[OK] $SERVICE"
    exit 0
else
    echo "[FAIL] $SERVICE"
    exit 2
fi
```

VARIABLES

éviter duplication, rendre configurable

CONDITIONS

transformer un fait en décision

EXIT CODE

fournir un état machine stable

QUESTION

Pourquoi un healthcheck ne doit-il pas automatiquement “réparer” nginx ?

Séparer détection et remédiation évite effets de bord et boucles cachées.

Codes de retour : une API minuscule mais puissante

Le shell, cron, systemd et le monitoring peuvent tous les exploiter.

0 — OK

Tout est dans les seuils attendus.

1 — WARNING

Dégradation ; service encore exploitable.

2 — CRITICAL

Service indisponible ou condition critique.

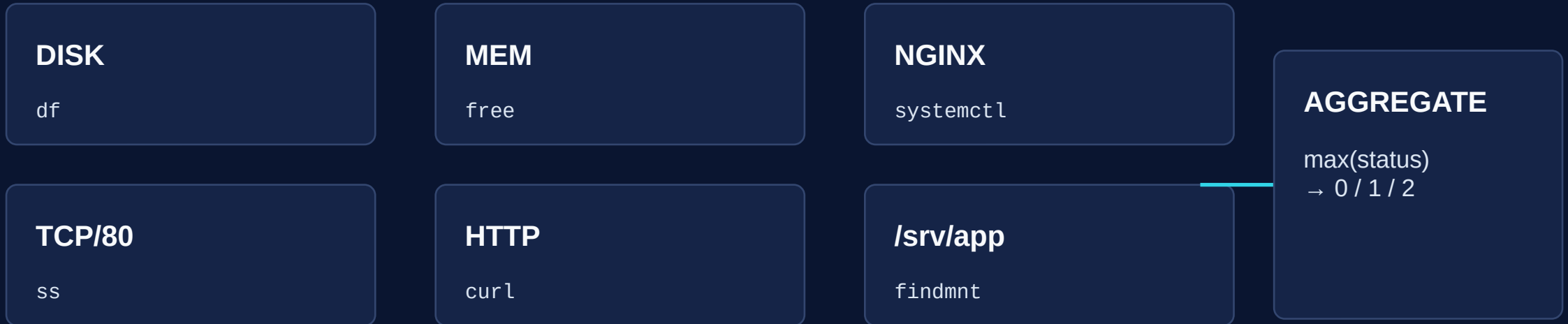
DÉMO

```
./novatech-healthcheck.sh  
echo $?
```

État global = le niveau le plus grave rencontré.

Architecture du healthcheck NovaTech

Mesurer plusieurs couches sans masquer les échecs.



Piège : `set -e` peut arrêter le script précisément quand un check retourne un échec attendu.

grep, sed, awk : extraire l'information utile

Vous n'avez pas besoin d'être expert awk ; vous devez savoir réduire un flux.

PIPELINE

```
grep -i error /var/log/syslog  
ss -lntp | grep ':80 '  
awk '{print $1}' fichier.txt  
sed 's/old/new/g' fichier.txt
```

grep

filtrer des lignes

awk

découper / calculer par colonnes

sed

transformer un flux texte

QUESTION

Pourquoi parser une sortie “humaine” peut être fragile dans un script ?

Format variable, locale, versions ; préférer sorties stables/JSON quand disponibles.

cron ou systemd timer ?

Les deux planifient ; systemd apporte intégration service/journal/dépendances.

CRON

- Simple et historique
- Syntaxe compacte
- Environnement réduit / différent du shell
- Logs à gérer explicitement
- Très bien pour tâches simples

SYSTEMD TIMER

- Unit service + unit timer
- Journal intégré
- Dépendances et politiques systemd
- Visibilité via list-timers
- Plus verbeux mais plus structuré

TP07 : cron obligatoire ; systemd timer en challenge.

Automatiser une tâche, c'est aussi automatiser ses sorties

Sans rotation, votre monitoring peut finir par provoquer la panne qu'il surveille.



VALIDATION

```
sudo logrotate -d /etc/logrotate.d/novatech-healthcheck
# observer avant :
sudo logrotate -f /etc/logrotate.d/novatech-healthcheck
```


logrotate : rotation, conservation et permissions

La syntaxe n'est utile que si vous testez ce qu'elle fera.

```
/etc/logrotate.d/  
novatech-healthcheck
```

```
/var/log/novatech/healthcheck.log {  
    daily  
    rotate 7  
    compress  
    missingok  
    notifempty  
    create 0640 root adm  
}
```

-d / debug

simule et explique sans rotation

-f / force

force la rotation pour le lab

create

recrée le fichier avec droits définis

Sauvegarder $\neq$ restaurer

Une archive non testée est une hypothèse de sauvegarde.



TP08 : BACKUP_RESTORE_PROOF doit être supprimé puis réellement restauré.

QUESTION

Pourquoi une sauvegarde sur le même disque protège-t-elle mal ?

Panne disque/VM, ransomware, suppression : même domaine de panne.

RPO et RTO : deux questions simples derrière une stratégie de reprise

Introduire le vocabulaire sans transformer ce module en cours PRA.

RPO — Recovery Point Objective

Combien de données puis-je accepter de perdre ?

Backup quotidien → jusqu'à ~24 h de données.

RTO — Recovery Time Objective

Combien de temps puis-je accepter d'être indisponible ?

Restore manuel local ≠ PRA automatisé.

Le lab teste la capacité technique à restaurer ; une vraie stratégie ajoute fréquence, hors-site, chiffrement, rétention et tests.

Baseline : réduire la surface et les privilèges

La sécurité n'est pas un "hardening final" : elle traverse tous les TPs.

PRIVILÈGES

sudo / comptes dédiés

SSH

clé / root interdit

SERVICES

n'exposer que nécessaire

PAQUETS

sources & updates

PORTS

savoir qui écoute

BACKUP

droits + restauration

QUESTION

Pourquoi chmod 777 "qui résout le problème" n'est-il pas une correction acceptable ?

Il élargit inutilement les droits et masque la cause exacte.

Un service peut être actif et pourtant malade

Caractérisiez la saturation avant de toucher à la configuration.

DIAGNOSTIC

```
uptime
free -h
df -hT
du -xhd1 /var | sort -h
ps aux --sort=-%cpu | head
ps aux --sort=-%mem | head
journalctl -p err --since today
```

CPU

load/process

RAM

available/swap

DISK

capacity/nodes

I/O

DÉMO LIVE

Créer un gros fichier dans /var/tmp → df/du → supprimer → vérifier. Ne pas le faire sur un vrai serveur de prod.

La méthode compte plus que la commande magique

Votre but : réduire l'espace des causes possibles avec des preuves.

1

REPRODUIRE

2

OBSERVER

3

DÉLIMITER

4

HYPOTHÈSE

5

TESTER

6

CORRIGER

7

VALIDER

8

DOCUMENTER

Interdits comme “diagnostic” : “j’ai rebooté”, “j’ai mis 777”, “j’ai désactivé le firewall”.

“Le site ne répond plus” : raisonnez verticalement

Client → socket → service → config → filesystem → permissions/resources

CLIENT / HTTP

`curl -v`

SOCKET / PORT

`ss -lntp`

SYSTEMD

`systemctl`

CONFIG

`nginx -t`

STORAGE

`findmnt / mount -a`

RIGHTS

`namei / ls -l / getfacl`

Commencez là où le symptôme est observable, puis descendez jusqu'à trouver la rupture.

INC-26001 — vous êtes l'équipe d'astreinte

Le formateur peut injecter plusieurs causes simultanées.

09:14 — INCIDENT CRITIQUE

L'application NovaTech n'est plus accessible après une intervention de maintenance. Le serveur répond au réseau, mais le service applicatif est indisponible ou dégradé.

PAS DE RÉINSTALL

réparer minimalement

PAS DE 777

cause précise

PREUVES

fait → hypothèse → test

RCA

documenter la cause

Pendant les 20 premières minutes, le formateur peut ne répondre que par des questions.

Ce qui sera évalué — 20 points

La démarche compte autant que le résultat technique.

TPs & validations

6 pts

QCM écrit

4 pts

Diagnostic incident

3 pts

Correction

3 pts

Validation

2 pts

Mini-RCA / oral

2 pts

Un serveur réparé “par chance” sans pouvoir expliquer la cause ne vaut pas un diagnostic démontré.

QUESTION

À l'oral : “Pourquoi cette commande ? Qu'est-ce que son résultat vous a permis d'exclure ?”

Préparez-vous à expliquer une ligne choisie au hasard dans votre RCA.

Le niveau attendu à la sortie

Vous n'avez pas besoin de tout connaître. Vous devez savoir chercher, prouver et changer proprement.

OBSERVER

avant de modifier

ADMINISTRER

sans root permanent

PERSISTER

filesystem/LVM/fstab

SERVIR

systemd & logs

AUTOMATISER

Bash & jobs

SÉCURISER

SSH & droits

RESTAURER

backup testé

DIAGNOSTIQUER

faits → RCA

Le Final Boss n'est pas la fin du cours : c'est la preuve que toutes les briques ont commencé à former un réflexe d'administrateur.